

VISUAL BASIC .NET

Unit-2: Data Types, Variables, Operators

Comprehensive Study Notes

Table of Contents

1. Data Types Overview

2. Built-in Value Types

3. Variables

4. Constants

5. Operators

6. Type Conversion

7. Boolean Expressions

8. Control Structures

9. Strings

10. Debugging

11. Quick Revision Points

Questions

1. Data Types Overview

Data types define the **type of data** that a variable can store. In Visual Basic, every variable must have a data type, which determines:

- The **size** of memory allocated
- The **range** of values it can hold
- The **operations** that can be performed on it

Value Types vs Reference Types

Aspect	Value Types	Reference Types
Storage	Stored on the stack	Stored on the heap
Content	Contains the actual data	Contains a reference (pointer) to the data
Copy Semantics	Copy of value is passed (independent copy)	Copy of reference is passed (same object)
Types	Boolean, Byte, Short, Integer, Long, Single, Double, Decimal, Char, Date, Structure	String, Array, Class, Object, Interface, Delegate
Example	Dim x As Integer = 10 Dim y = x: y = 20 (x remains 10)	Dim list1 As New List(Of Integer) Dim list2 = list1: list2.Add(20) (list1 also has 20)

2. Built-in Value Types

Complete Data Type Table

Type	Alias	Size	.NET Type	Range	Example
Boolean	Boolean	1 byte	System.Boolean	True or False	Dim isActive As Boolean = True
Byte	Byte	1 byte	System.Byte	0 to 255	Dim b As Byte = 255
SByte	SByte	1 byte	System.SByte	-128 to 127	Dim sb As SByte = -100
Short	Short	2 bytes	System.Int16	-32,768 to 32,767	Dim s As Short = 1000
UShort	UShort	2 bytes	System.UInt16	0 to 65,535	Dim us As UShort = 65000
Integer	Integer	4 bytes	System.Int32	-2.14x10 ⁹ to 2.14x10 ⁹	Dim i As Integer = 1000000
UInteger	UInteger	4 bytes	System.UInt32	0 to 4.29x10 ⁹	Dim ui As UInteger = 4000000000
Long	Long	8 bytes	System.Int64	-9.22x10 ¹⁸ to 9.22x10 ¹⁸	Dim l As Long = 10000000000
ULong	ULong	8 bytes	System.UInt64	0 to 1.84x10 ¹⁹	Dim ul As ULong = 18446744073709551615
Single	Single	4 bytes	System.Single	+/-3.4x10 ³⁸ (7 digit precision)	Dim f As Single = 3.14159
Double	Double	8 bytes	System.Double	+/-1.7x10 ³⁰⁸ (15-16 digit precision)	Dim d As Double = 3.14159265359
Decimal	Decimal	16 bytes	System.Decimal	+/-7.9x10 ²⁸ (28-29 digit precision)	Dim m As Decimal = 3.14159265358979D
Char	Char	2 bytes	System.Char	Unicode character (0 to	Dim c As Char = "A"c

				65,535)	
String	String	Variable	System.String	Unicode text (up to 2GB)	<code>Dim s As String = "Hello"</code>
Date	Date	8 bytes	System.DateTime	Jan 1, 0001 to Dec 31, 9999	<code>Dim d As Date = #2026-07-11#</code>
Object	Object	4 bytes	System.Object	Any type	<code>Dim o As Object = 10</code>

Reference Types

Type	Description	Example
String	Sequence of Unicode characters (immutable)	<code>Dim s As String = "Hello"</code>
Array	Fixed-size collection of elements of the same type	<code>Dim arr() As Integer = {1, 2, 3}</code>
Class	Blueprint for creating objects with properties and methods	<code>Dim obj As New Customer()</code>
Object	Base type for all .NET types	<code>Dim o As Object = "Anything"</code>
Interface	Contract defining methods/properties that implementing classes must provide	<code>Implements IDisposable</code>
Delegate	Type-safe function pointer for event handling	<code>Delegate Sub MyDelegate()</code>

3. Variables

Variable Declaration Syntax

```
' Basic Syntax
Dim variableName As DataType
```

```
' Examples
Dim age As Integer
Dim name As String
Dim salary As Double
Dim isActive As Boolean
```

Variable Initialization

```
' Declaration and initialization in one line
Dim age As Integer = 25
Dim name As String = "John Doe"
Dim salary As Double = 50000.50
Dim isActive As Boolean = True

' Declaration first, then initialization
Dim count As Integer
count = 10

' Multiple variables in one line
Dim x As Integer = 10, y As Integer = 20, z As Integer = 30

' Variables with same type
Dim a, b, c As Integer ' All are Integer
a = 1
b = 2
c = 3

' Type Inference (Option Infer On)
Dim number = 10        ' Infer As Integer
Dim text = "Hello"     ' Infer As String
Dim price = 99.99      ' Infer As Double
Dim flag = True        ' Infer As Boolean
```

Variable Naming Rules

Rule	Explanation	Good Examples	Bad Examples

Must begin with letter	Variable name must start with a letter or underscore	<code>name</code> , <code>_count</code>	<code>1name</code> , <code>@count</code>
Can contain letters, digits, underscore	Only alphanumeric and underscore allowed	<code>myName</code> , <code>total_amount</code>	<code>my-name</code> , <code>total\$</code>
Cannot be a keyword	Cannot use reserved words	<code>customerName</code>	<code>If</code> , <code>For</code> , <code>Dim</code>
Case-insensitive	VB is not case-sensitive	<code>Name</code> and <code>name</code> are same	-
Maximum length	255 characters maximum	<code>customerFirstName</code>	Very long names

Variable Scope and Lifetime

Scope Level	Where Declared	Accessible	Lifetime
Block Level	Inside a block (If, For, Using, etc.)	Only within that block	Declaration to end of block
Procedure Level	Inside a Sub or Function	Only within that procedure	Declaration to end of procedure
Module/Class Level	Inside a Class/Module, outside any procedure	All procedures in the class/module	Until object is destroyed
Namespace/Project Level	At namespace level (outside any class)	All classes in the namespace	Lifetime of the application

Scope Examples

```
' BLOCK LEVEL SCOPE
If x > 10 Then
    Dim temp As Integer = x * 2 ' Block scope
    MessageBox.Show(temp.ToString())
End If
' temp is NOT accessible here

' PROCEDURE LEVEL SCOPE
Private Sub Calculate()
    Dim result As Integer = 10 + 20 ' Procedure scope
    MessageBox.Show(result.ToString())
End Sub
' result is NOT accessible here

' MODULE / CLASS LEVEL SCOPE
```

```
Public Class Form1
    Private counter As Integer = 0 ' Class scope

    Private Sub Button1_Click( ... ) Handles Button1.Click
        counter += 1 ' Accessible here
    End Sub

    Private Sub Button2_Click( ... ) Handles Button2.Click
        MessageBox.Show(counter.ToString()) ' Accessible here
    End Sub
End Class
```

Access Modifiers for Variables

Modifier	Description	Scope
Public	Accessible from anywhere	Whole application
Private	Accessible only within the same class/module	Class/Module only
Protected	Accessible within class and derived classes	Class and subclasses
Friend	Accessible within the same assembly	Assembly (Project)
Protected Friend	Protected OR Friend	Assembly or subclass
Private Protected	Private AND Protected	Same class or subclass in same assembly

4. Constants

Constants are variables whose values **cannot be changed** after they are declared. They are useful for values that remain the same throughout the program.

Constant Declaration Syntax

' Basic Syntax

```
Const constantName As DataType = value
```

' Examples

```
Const PI As Double = 3.14159
```

```
Const MAX_VALUE As Integer = 100
```

```
Const APP_NAME As String = "My Application"
```

```
Const DAYS_IN_WEEK As Integer = 7
```

Constants vs Variables

Aspect	Constants	Variables
Value	Cannot be changed	Can be changed
Declaration	Const keyword	Dim keyword
Initialization	Must be initialized at declaration	Can be initialized later
Memory	May be optimized by compiler	Allocated at runtime
Use Case	Fixed values (PI, days in week)	Changing values (counter, name)

Benefits of Using Constants

1. **Readability** - Meaningful names instead of magic numbers
2. **Maintainability** - Change in one place, affects all uses
3. **Error Prevention** - Cannot accidentally change value
4. **Performance** - May be optimized by compiler

5. Operators

Arithmetic Operators

Operator	Description	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division (Double)	10 / 3	3.33333333333333
\	Integer Division	10 \ 3	3
Mod	Modulus (Remainder)	10 Mod 3	1
^	Exponentiation	10 ^ 2	100

Operator Precedence (Arithmetic)

1. ^ (Exponentiation) - Highest
2. * and / (Multiplication and Division)
3. \ (Integer Division)
4. Mod (Modulus/Remainder)
5. + and - (Addition and Subtraction) - Lowest

Comparison Operators

Operator	Description	Example	Result
=	Equal to	5 = 5	True
<>	Not equal to	5 <> 10	True
<	Less than	5 < 10	True
>	Greater than	10 > 5	True
≤	Less than or equal to	10 ≤ 10	True
≥	Greater than or equal to	10 ≥ 5	True
Is	Object identity (same reference)	obj1 Is obj2	True/False

IsNot	Object inequality (different reference)	obj1 IsNot obj2	True/False
Like	Pattern matching	"Hello" Like "H*"	True

Logical Operators

Operator	Description	Truth Table
And	Both must be True	T+T=T, T+F=F, F+T=F, F+F=F
Or	Either can be True	T+T=T, T+F=T, F+T=T, F+F=F
Not	Reverses the value	Not T = F, Not F = T
Xor	Exactly one must be True	T+T=F, T+F=T, F+T=T, F+F=F
AndAlso	Short-circuit AND (evaluates right only if left is True)	Same as And but short-circuits
OrElse	Short-circuit OR (evaluates right only if left is False)	Same as Or but short-circuits

Concatenation Operators

Operator	Description	Example	Result
&	String concatenation (preferred)	"Hello" & " " & "World"	"Hello World"
+	String concatenation (alternative)	"Hello" + "World"	"HelloWorld"

Assignment Operators

Operator	Description	Example	Equivalent To
=	Simple assignment	x = 10	-
+=	Add and assign	x += 5	x = x + 5
-=	Subtract and assign	x -= 5	x = x - 5
*=	Multiply and assign	x *= 5	x = x * 5
/=	Divide and assign	x /= 5	x = x / 5
\=	Integer divide and assign	x \= 3	x = x \ 3
^=	Exponentiate and assign	x ^= 2	x = x ^ 2
&=	Concatenate and assign	s &= "World"	s = s & "World"

<<=	Left shift and assign	x <<= 2	x = x << 2
>>=	Right shift and assign	x >>= 2	x = x >> 2

Shift Operators

Operator	Description	Example	Result
<<	Left shift (multiply by 2^n)	5 << 2	20
>>	Right shift (divide by 2^n)	20 >> 2	5

Complete Operator Precedence Table

Level	Category	Operators
1	Primary	. (member access), () (method call), New , AddressOf
2	Unary	+ (unary plus), - (unary minus), Not , Is , IsNot
3	Exponentiation	^
4	Multiplication/Division	*, /
5	Integer Division	\
6	Modulus	Mod
7	Addition/Subtraction	+, -
8	Shift	<< , >>
9	Comparison	= , <> , < , > , ≤ , ≥ , Like , TypeOf
10	Logical AND	And , AndAlso
11	Logical OR	Or , OrElse , Xor
12	Concatenation	& , +

6. Type Conversion

Implicit Casting (Widening)

Automatic conversion by the compiler. No data loss (safe conversion). Smaller type to larger type.

```
Dim intVal As Integer = 10
Dim longVal As Long = intVal      ' Integer ? Long (Automatic)
Dim singleVal As Single = longVal ' Long ? Single (Automatic)
Dim dblVal As Double = singleVal  ' Single ? Double (Automatic)
```

From Type	To Type	Safe?
Byte	Short, Integer, Long, Single, Double, Decimal	Yes
Short	Integer, Long, Single, Double, Decimal	Yes
Integer	Long, Single, Double, Decimal	Yes
Long	Single, Double, Decimal	Yes
Single	Double	Yes
Char	String	Yes
Any Type	Object	Yes

Explicit Casting (Narrowing) - Conversion Functions

Manual conversion using conversion functions. Potential data loss. Larger type to smaller type.

Function	Converts To	Example
CInt()	Integer	CInt(10.7) = 11 (rounds)
CStr()	String	CStr(10.5) = "10.5"
Cdbl()	Double	Cdbl("10.5") = 10.5
CBool()	Boolean	CBool(1) = True
CDate()	Date	CDate("2026-07-11")
CByte()	Byte	CByte(255)
CChar()	Char	CChar("A")

CDec()	Decimal	CDec("10.5")
CLng()	Long	CLng(1000000)
CObj()	Object	CObj(10)
CShort()	Short	CShort(1000)
CSng()	Single	CSng("10.5")
CType()	Any type	CType(obj, Integer)

Using Conversion Functions

```
' Explicit Casting Examples
Dim dblVal As Double = 10.7
Dim intVal As Integer = CInt(dblVal) ' 10.7 ? 11 (Rounded)
Dim strVal As String = "123"
Dim numVal As Integer = CInt(strVal) ' "123" ? 123
Dim boolVal As Boolean = CBool(1) ' 1 ? True
Dim dateVal As Date = CDate("2026-07-11")

' Using CType
Dim result As Integer = CType(dblVal, Integer)

' TryParse (safe conversion without exceptions)
Dim input As String = "123"
Dim parsed As Integer
If Integer.TryParse(input, parsed) Then
    MessageBox.Show("Parsed: " & parsed.ToString())
Else
    MessageBox.Show("Invalid input")
End If
```

Option Strict and Option Explicit

```
' Option Explicit: Forces explicit variable declaration
' Option Strict: Enforces strict type checking

' File header settings
Option Explicit On ' Must declare variables before using
Option Strict On ' Strict type checking (recommended)
Option Infer On ' Allow type inference
```

Formatting Functions

Specifier	Description	Example	Output
-----------	-------------	---------	--------

F or f	Fixed point	<code>1234.56.ToString("F2")</code>	1234.56
N or n	Number with commas	<code>1234.56.ToString("N2")</code>	1,234.56
C or c	Currency	<code>1234.56.ToString("C")</code>	\$1,234.56
P or p	Percentage	<code>0.1234.ToString("P2")</code>	12.34%
E or e	Exponential	<code>1234.56.ToString("E2")</code>	1.23E+003
G or g	General	<code>1234.56.ToString("G")</code>	1234.56
D or d	Decimal (integers)	<code>123.ToString("D6")</code>	000123
X or x	Hexadecimal	<code>255.ToString("X")</code>	FF

7. Boolean Expressions

Boolean expressions evaluate to either **True** or **False**. They are used in conditional statements and loops to control program flow.

Logical Operators - Truth Tables

AND (Both must be True)

A	B	A And B
True	True	True
True	False	False
False	True	False
False	False	False

OR (Either can be True)

A	B	A Or B
True	True	True
True	False	True
False	True	True
False	False	False

NOT (Reverses the value)

A	Not A
True	False
False	True

XOR (Exactly one must be True)

A	B	A Xor B
True	True	False
True	False	True
False	True	True
False	False	False

AndAlso - Short-circuit AND (evaluates right side ONLY if left is True)

OrElse - Short-circuit OR (evaluates right side ONLY if left is False)

Boolean Expression Examples

' Relational operators

```
Dim a As Integer = 10, b As Integer = 20
Dim result1 As Boolean = (a = b)      ' False
Dim result2 As Boolean = (a < b)      ' True
Dim result3 As Boolean = (a <> b)     ' True
```

' Logical operators

```
Dim x As Boolean = True, y As Boolean = False
Dim r1 As Boolean = x And y          ' False
Dim r2 As Boolean = x Or y           ' True
Dim r3 As Boolean = Not x            ' False
Dim r4 As Boolean = x Xor y          ' True
```

' Short-circuit operators

```
Dim count As Integer = 0
' AndAlso - second condition NOT evaluated if first is False
If count > 0 AndAlso (10 \ count) > 2 Then
    ' Safe from divide-by-zero
End If
```

' Like operator - pattern matching

```
Dim str As String = "Hello"
If str Like "H*" Then MessageBox.Show("Starts with H")
If str Like "?ello" Then MessageBox.Show("5 chars, ends with ello")
If str Like "[A-H]*" Then MessageBox.Show("Starts with A-H")
```

8. Control Structures

If Statement

```
' Syntax:
' If condition Then
'     ' Code when condition is True
' ElseIf anotherCondition Then
'     ' Code when another condition is True
' Else
'     ' Code when all conditions are False
' End If

' Examples:
Dim age As Integer = 18

' Simple If
If age ≥ 18 Then
    MessageBox.Show("You are an adult.")
End If

' If-Else
If age ≥ 18 Then
    MessageBox.Show("You can vote.")
Else
    MessageBox.Show("You are too young to vote.")
End If

' ElseIf (multiple conditions)
Dim marks As Integer = 85
If marks ≥ 90 Then
    MessageBox.Show("Grade A")
ElseIf marks ≥ 80 Then
    MessageBox.Show("Grade B")
ElseIf marks ≥ 70 Then
    MessageBox.Show("Grade C")
ElseIf marks ≥ 60 Then
    MessageBox.Show("Grade D")
Else
    MessageBox.Show("Grade F")
End If

' Nested If
Dim isLoggedIn As Boolean = True
Dim isAdmin As Boolean = True

If isLoggedIn Then
    If isAdmin Then
        MessageBox.Show("Welcome Admin!")
    Else
```

```
        MessageBox.Show("Welcome User!")
    End If
Else
    MessageBox.Show("Please login.")
End If
```

Select Case Statement

```
' Syntax:
' Select Case expression
'     Case value1
'         ' Code for value1
'     Case value2 To value3
'         ' Code for range
'     Case Is > value
'         ' Code with condition
'     Case Else
'         ' Default code
' End Select

' Examples:
Dim day As Integer = 3
Dim dayName As String = ""

Select Case day
    Case 1
        dayName = "Monday"
    Case 2
        dayName = "Tuesday"
    Case 3
        dayName = "Wednesday"
    Case 4
        dayName = "Thursday"
    Case 5
        dayName = "Friday"
    Case 6 To 7
        dayName = "Weekend"
    Case Else
        dayName = "Invalid day"
End Select

' Select Case with multiple values and conditions
Dim score As Integer = 85
Select Case score
    Case 90 To 100
        MessageBox.Show("Excellent!")
    Case 75 To 89
        MessageBox.Show("Good Job!")
    Case 50, 60, 70
        MessageBox.Show("Passing")
    Case Is < 50
```

```
        MessageBox.Show("Needs Improvement")
    End Select
```

For Loop

```
' Syntax:
' For counter As Integer = start To end Step step
'     ' Loop body
' Next counter

' Forward loop
For i As Integer = 1 To 5
    MessageBox.Show("Iteration: " & i.ToString())
Next

' Reverse loop with Step
For i As Integer = 10 To 1 Step -1
    Debug.WriteLine(i.ToString())
Next

' For Each loop (collections)
Dim numbers() As Integer = {1, 2, 3, 4, 5}
Dim total As Integer = 0
For Each num As Integer In numbers
    total += num
Next
MessageBox.Show("Sum: " & total.ToString())

' Step by 2 (even numbers)
For i As Integer = 0 To 10 Step 2
    Debug.WriteLine(i.ToString()) ' 0, 2, 4, 6, 8, 10
Next
```

While Loop

```
' Syntax:
' While condition
'     ' Loop body
' End While

Dim counter As Integer = 1
While counter ≤ 5
    Debug.WriteLine("Count: " & counter.ToString())
    counter += 1
End While
```

Do Loop Variations

```
' Do While (condition checked BEFORE execution)
Dim i As Integer = 1
Do While i ≤ 5
    Debug.WriteLine(i.ToString())
    i += 1
Loop

' Do Until (loop WHILE condition is False)
Dim j As Integer = 1
Do Until j > 5
    Debug.WriteLine(j.ToString())
    j += 1
Loop

' Do ... Loop While (executes AT LEAST ONCE, checks after)
Dim k As Integer = 1
Do
    Debug.WriteLine(k.ToString())
    k += 1
Loop While k ≤ 5

' Do ... Loop Until
Dim m As Integer = 1
Do
    Debug.WriteLine(m.ToString())
    m += 1
Loop Until m > 5
```

With End With

```
' With statement simplifies repeated property access
With Button1
    .Text = "Click Me"
    .BackColor = Color.Blue
    .ForeColor = Color.White
    .Font = New Font("Arial", 12, FontStyle.Bold)
    .Size = New Size(100, 30)
    .Location = New Point(50, 50)
End With
```

Exit, Continue, Stop, End

```
' EXIT STATEMENT - Immediately exits the loop/procedure
For i As Integer = 1 To 10
    If i = 5 Then
        Exit For ' Exit the loop when i = 5
    End If
    Debug.WriteLine(i.ToString())
Next
```

```
' Exit Do, Exit While, Exit Sub, Exit Function also available

' CONTINUE STATEMENT - Skips current iteration, moves to next
For i As Integer = 1 To 10
    If i Mod 2 = 0 Then
        Continue For ' Skip even numbers
    End If
    Debug.WriteLine("Odd: " & i.ToString())
Next

' STOP - Pauses execution (debug mode)
' Stop ' Uncomment to pause debugger here

' END - Terminates the application immediately
' If emergency Then End
```

9. Strings

Strings are **sequences of characters** used to represent text. In Visual Basic, strings are **reference types** and are **immutable** (cannot be changed once created).

String Methods and Properties

Method/Property	Description	Example	Result
Length	Returns number of characters	"Hello".Length	5
ToUpper()	Converts to uppercase	"hello".ToUpper()	"HELLO"
ToLower()	Converts to lowercase	"HELLO".ToLower()	"hello"
Trim()	Removes leading/trailing spaces	" Hello ".Trim()	"Hello"
Substring(start)	Extracts from position	"Hello".Substring(2)	"llo"
Substring(start, len)	Extracts specific length	"Hello".Substring(0, 3)	"Hel"
Contains(text)	Checks if substring exists	"Hello".Contains("ell")	True
IndexOf(text)	Finds position of substring	"Hello".IndexOf("l")	2
LastIndexOf(text)	Finds last occurrence	"Hello".LastIndexOf("l")	3
Replace(old, new)	Replaces substrings	"Hello".Replace("l", "x")	"Hexxo"
Split()	Splits into array	"a,b,c".Split(", "c)	{"a","b","c"}

<code>StartsWith(text)</code>	Checks beginning	<code>"Hello".StartsWith("He")</code>	True
<code>EndsWith(text)</code>	Checks ending	<code>"Hello".EndsWith("lo")</code>	True
<code>IsNullOrEmpty()</code>	Checks if null or empty	<code>String.IsNullOrEmpty(s)</code>	True/False
<code>IsNullOrWhiteSpace()</code>	Checks if whitespace	<code>String.IsNullOrWhiteSpace(s)</code>	True/False
<code>PadLeft(width)</code>	Left pads with spaces	<code>"5".PadLeft(3)</code>	" 5"
<code>PadRight(width)</code>	Right pads with spaces	<code>"5".PadRight(3)</code>	"5 "
<code>Insert(pos, text)</code>	Inserts text at position	<code>"Hello".Insert(0, "Hi ")</code>	"Hi Hello"
<code>Remove(start, len)</code>	Removes characters	<code>"Hello".Remove(2,2)</code>	"Heo"
<code>Compare(str1, str2)</code>	Compares two strings	<code>String.Compare("a", "b")</code>	-1
<code>Concat(str1, str2)</code>	Concatenates strings	<code>String.Concat("a", "b")</code>	"ab"

String Examples

```
' String declaration and initialization
Dim greeting As String = "Hello World"
Dim empty As String = ""
Dim nullStr As String = Nothing

' String joining and appending
Dim first As String = "John"
Dim last As String = "Doe"
Dim fullName As String = first & " " & last ' "John Doe"

' Using StringBuilder for efficiency with many concatenations
Dim sb As New System.Text.StringBuilder()
sb.Append("Line 1")
sb.AppendLine()
sb.Append("Line 2")
Dim result As String = sb.ToString()

' String methods in action
```



```
Dim text As String = " Visual Basic .NET "  
MessageBox.Show(text.Trim())           ' "Visual Basic .NET"  
MessageBox.Show(text.ToUpper())        ' " VISUAL BASIC .NET "  
MessageBox.Show(text.Substring(2, 6))   ' "Visual"  
Dim words() As String = text.Trim().Split(" ")  
  
' String comparison  
Dim str1 As String = "Hello"  
Dim str2 As String = "hello"  
If String.Compare(str1, str2, True) = 0 Then  
    MessageBox.Show("Case-insensitive match")  
End If  
  
' Formatting strings  
Dim price As Double = 1234.56  
Dim formatted As String = String.Format("Price: {0:C}", price)  
MessageBox.Show(formatted) ' "Price: $1,234.56"
```

10. Debugging Techniques for Loops

Technique	Description
Breakpoints in Loops	Set breakpoint inside loop body. Use conditional breakpoints to pause at specific iterations (e.g., break when <code>i = 5</code>).
Watch Variables	Add loop counter to Watch window. Monitor variables that change in the loop. Watch expressions like <code>i * 2</code> .
Locals Window	Shows all local variables in the loop. See values change with each iteration. Edit values during debugging.
Output Window	Use <code>Debug.WriteLine()</code> to output values. Trace execution flow. Log loop iterations.

Common Loop Problems

Problem	Description	Solution
Infinite Loop	Condition never becomes false	Ensure loop variable is updated correctly
Off-by-One Error	Loop runs too many or too few times	Check start and end values carefully
Index Out of Bounds	Accessing array beyond length	Use <code>array.Length - 1</code> as upper bound
Logic Errors	Wrong calculations inside loop	Trace values using <code>Debug.WriteLine()</code>

Debugging Example

```
Imports System.Diagnostics ' For Debug.WriteLine

Private Sub DebuggingLoopExample()
    Dim numbers() As Integer = {10, 20, 30, 40, 50}
    Dim total As Integer = 0

    ' Set a breakpoint here and watch 'i' and 'total'
    For i As Integer = 0 To numbers.Length - 1
        total += numbers(i)
        Debug.WriteLine("i=" & i & ", numbers(i)=" & numbers(i) &
            ", total=" & total)
    Next
End Sub
```

```
        MessageBox.Show("Final Total: " & total.ToString())  
    End Sub
```

11. Quick Revision Points

Topic	Key Points
Data Types	Boolean, Byte, Integer, Long, Single, Double, Decimal, String, Date, Object
Variables	Declared with <code>Dim</code> , can be initialized at declaration
Constants	Declared with <code>Const</code> , values cannot be changed
Arithmetic	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>\</code> (integer division), <code>Mod</code> , <code>^</code> (power)
Precedence	<code>^</code> ? <code>*</code> <code>/</code> <code>/</code> ? <code>\</code> ? <code>Mod</code> ? <code>+</code> <code>/</code> <code>-</code>
Assignment	<code>=</code> operator, compound operators (<code>+=</code> , <code>-=</code> , etc.)
Casting	<code>CInt()</code> , <code>Cdbl()</code> , <code>CStr()</code> , <code>CType()</code> , <code>TryParse()</code>
Strings	Immutable, use <code>&</code> for concatenation, many methods available
Formatting	<code>ToString()</code> with format specifiers
Booleans	True/False values, relational and logical operators
If Statement	<code>If ... Then ... ElseIf ... Else ... End If</code>
Select Case	Multiple branches, ranges, conditions, multiple values
For Loop	<code>For ... To ... Step ... Next</code>
For Each	Iterate through collections
While Loop	<code>While ... End While</code>
Do Loop	<code>Do While / Until ... Loop</code> or <code>Do ... Loop While / Until</code>
Exit	<code>Exit For</code> , <code>Exit Do</code> , <code>Exit Sub</code> , <code>Exit Function</code>
Continue	<code>Continue For</code> , <code>Continue Do</code>
Debugging	Breakpoints, Watch windows, Locals window, Output window

Questions

1. What are data types in VB.NET? Explain value types and reference types with examples.

- List all built-in value types with their size, range, and .NET type
- Explain memory allocation for value types vs reference types

2. Explain variable declaration, initialization, and scope in VB.NET.

- Discuss block, procedure, module, and namespace level scopes with examples
- Explain access modifiers: Public, Private, Protected, Friend
- What are the variable naming rules?

3. What are constants? How do they differ from variables?

- Syntax and examples of constants
- Benefits of using constants in programming

4. Explain all types of operators in VB.NET with examples.

- Arithmetic, comparison, logical, concatenation, assignment, and shift operators
- Operator precedence with a complete precedence table

5. What is type conversion? Explain implicit and explicit casting.

- List all conversion functions: CInt, CStr, CDbI, CBool, CDate, CType
- Explain widening and narrowing conversions
- What is Option Strict and Option Explicit?

6. Explain control structures in VB.NET.

- If, If-Else, Elseif, nested If with examples
- Select Case with ranges, conditions, and multiple values

7. Explain all looping constructs in VB.NET with examples.

- For Next, For Each, While, Do While, Do Until, Do...Loop While, Do...Loop Until
- Explain Exit and Continue statements

8. Write a VB.NET program that demonstrates:

- Variable declarations of different data types
- Arithmetic operations with all operators
- Type conversion using CInt, CDbI, CStr
- If-Else and Select Case for decision making
- For and Do loops for iteration

9. Explain string handling in VB.NET.

- List and explain at least 15 string methods with examples
- String concatenation, StringBuilder, formatting

10. Discuss debugging techniques for programs with loops.

- Breakpoints, Watch window, Locals window, Output window
- Common loop problems and solutions